PORTFOLIO

AWS/Azure 멀티 클라우드부터 베어메탈 Kubernetes 보안 까지 이해하는 엔지니어, 오지민입니다.

Github
wlals2

blog
jiminhome.shop

Phone
010-8278-3213

Email
fw4568@gmail.com

보유 기술 및 엔지니어 역량

Infrastructure & Cloud

- **Kubernetes:**
  - **EKS & Bare-metal(v1.31)** 구축, Karpenter 오토 스케일링
  - **TroubleShooting:** HPA와 ArgoCD 충돌시 ignoreDifferences 활용, Karpetner 태그 누락 (SQS) 해결
- **Cloud Platform**
  - **AWS:** 고가용성 과 최소 권한 원칙의 아키텍처 구성 (ALB, IAM Rule 등)
  - **Azure:** DR 구성을 위한 Cold Standby VM 및 Vnet아키텍처
  - **GCP & NCP** : 3tier Architecture 구축 , Big Query 경험
- **Infrastructure as Code:**
  - **Terraform**: Multi-Cloud 프로비저닝, State Locking 및 모듈화
  - **Vagrant** : Venton VMware 프로비저닝

DevSecOps & CI/CD (보안 & 파이프라인)

- **CI/CD Pipeline:**
  - **GitHub Actions:** Self-Hosted Runner 구축 (폐쇄망 환경 배포), Maven 캐싱 최적화(빌드 시간 단축)
  - **ArgoCD:** App of Apps 패턴, Sync Policy 관리, SSOT 구현
  - **Argo Rollouts:** Canary 배포 전략 및 Traffic Routing 제어
- **Security Engineering:**
  - **Falco:** 런타임 위협 탐지 (Shell 접속, 민감 파일 접근)
  - **Trivy:** 컨테이너 이미지 취약점 스캔 (CI 단계 통합)

Network & Observability (네트워크 & 관제)

- **Zero Trust Network:**
  - **Cilium (eBPF):** L3/L4 NetworkPolicy 최적화 (iptables 대체)
  - **Cloudflare:** Tunnel & WARP 기반 사내망(Bare-metal) 보안 접속 설계
- **Service Mesh:**
  - **Istio:** Traffic Shifting, mTLS 도입 및 JDBC 호환성 이슈 해결
- **Observability (PLG Stack):**
  - **Hubble UI:** L7 네트워크 패킷 흐름(Flow) 시각화
  - Prometheus, Grafana, Loki, Kiali (Service Mesh 토폴로지)

"최고의 기술보다, 최적의 기술을 설계하는 엔지니어 오지민 입니다."

장애 앞에서는 **데이터(Log)**로 말하고, 운영 앞에서는 **효율(Cost)**로 증명하겠습니다.

겉핥기식 구현이 아닌, **패킷(Packet)과 커널(Kernel)까지 들여다보는 집요함**으로 팀에 기여하겠습니다.

Project: 멀티클라우드 DR 고가용성 인프라

프로젝트 목표

- AWS 내에서 **고가용성 서비스** & Azure 로 **DR 체계 구축**

핵심 성과

- **고가용성 (99.9%)** : EKS Multi-AZ + Karpenter 자동 스케일링 으로 트래픽 유연 대응
- **DR System(RTO 30min)**:AWS 장애시 Azure로 자동 Failover
- **비용 최적화 ($10/mo)** : Cold standby + backup restore DR 비용 최적화
- **GitOps & IaC** : 인프라 코드화 & Canary 로 무중단 배포
- **Cost Visibility:** OpenCost 도입으로 실시간 비용 추적 및 최적화

기술 스택

Project: 제로 트러스트 Bare Metal K8s

프로젝트 목표

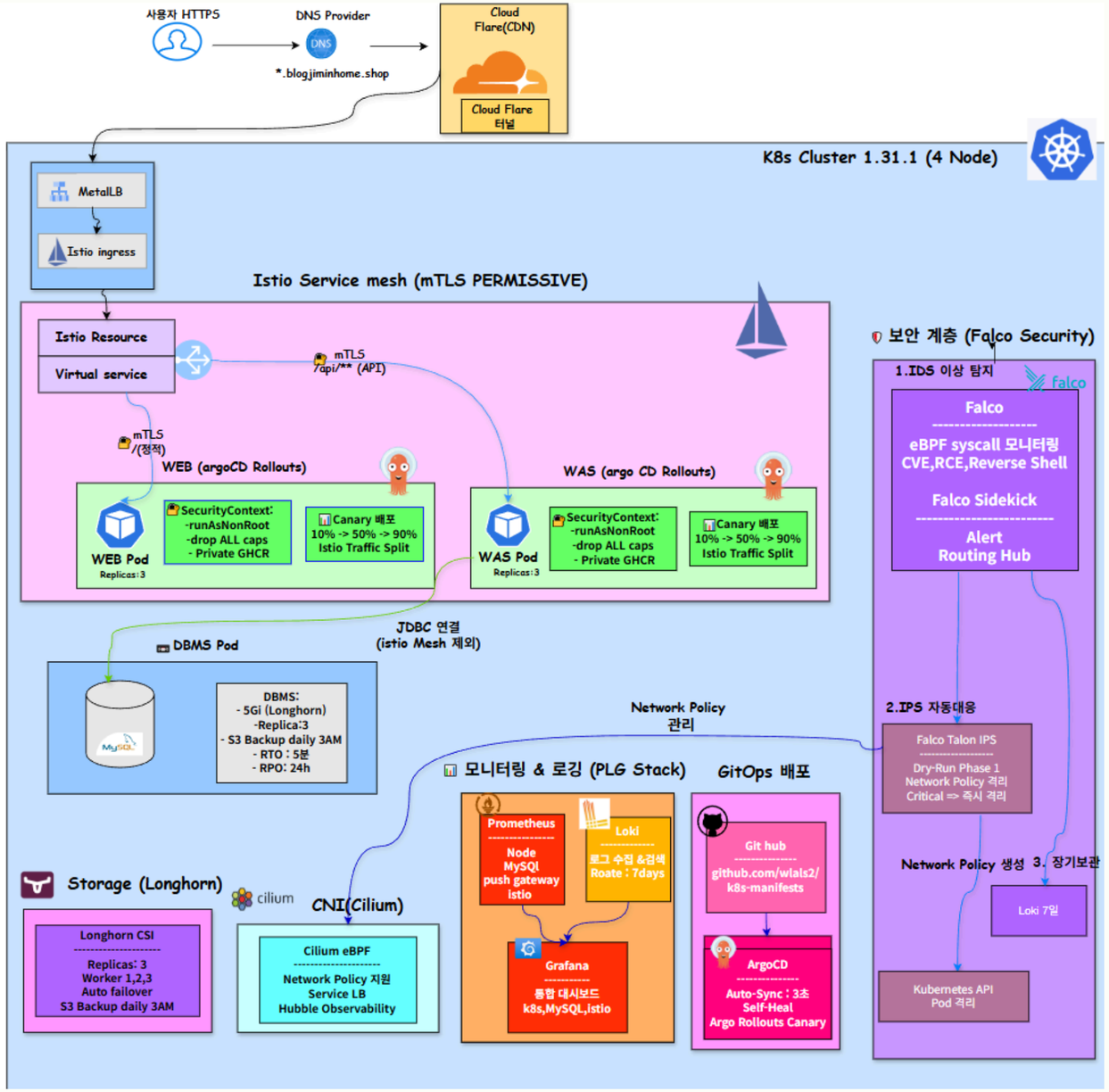
- 물리 서버 환경에서 **ZeroTrust** 및 **DevSecOps** 구현

핵심 성과

- **업 타임 (가동률 99.9%):** 60일 간 다운 타임 없는 블로그 운영 **(ArgoRollouts 무중단 배포)**
- **DevSecOps**: **L3/L4(Cilium)** 정책, **L7(Istio)** 암호화 , **Runtime(Falco)**탐지 , **IPS**(Falco-Talon)
- **상세 모니터링** : Hubble UI 통한 **L7 네트워크 패킷흐름 100% 가시화**
- **네트워크 최적화 (iptables rule 90% 제거)**: iptables를 EBPF로 대체하여 네트워크 처리량 및 Latency 개선

기술 스택

Project: Zero Trust Bare Metal K8s



Zero Trust architecture 구현 (계층별 방어)

- **Defense in Depth (심층 방어):**
  - **L3/L4 (Kernel):** Cilium eBPF로 iptables 없이 고성능 패킷 필터링
  - **L7 (Application):** Istio mTLS(PERMISSIVE)로 Pod 간 통신 암호화
- **Runtime Security:**
  - Falco를 통해 쉘 접속, 민감 파일 접근 등 **이상 행동 실시간 탐지**
- **효과 :** 네트워크와 런타임 계층을 모두 방어하여 **공격 표면 최소화**

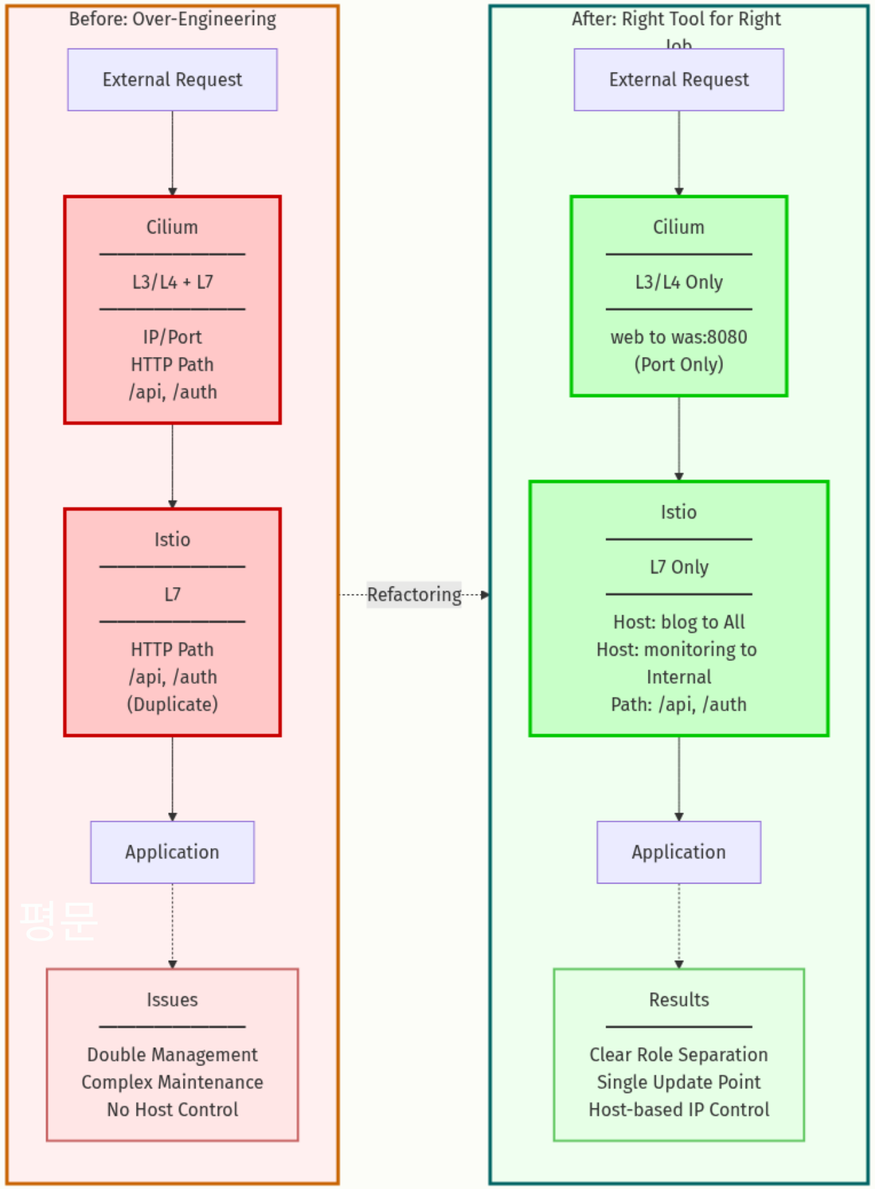
Full-Stack Observability

- **L7 Deep Visibility :**
  - **Hubble UI**를 통해 커널 레벨에서 HTTP/DNS 패킷 흐름 **100% 시각화.**
  - **Kiali로** mTLS 트래픽 암호화 여부 및 서비스 의존성 실시간 확인
- **Integrated Monitoring:**
  - **Metrics:** Prometheus + Grafana **(시스템 리소스)**
  - **Logs:** Loki + Alloy로 장애 발생 서버의 로그를 중앙에서 즉시 조회
  - **성과:** 장애 발생 시 로그와 트래픽을 연계하여 5분 내 근본 원인(RCA) 파악.

DevSecOps Pipeline

- **Shift Left (build) :**
  - **Trivy:** CI 단계에서 컨테이너 이미지 취약점(CVE) 스캔 및 배포 차단
- **Automated Response (Runtime) :**
  - **Falco-Talon:** 위협 탐지시 즉시 해당 pod를 네트워크 격리 시켜 공격 무효화
  - **Cilium Network Policy:** L3/L4 정책을 코드로 관리 보안 설정을 불변성 보장

Deep Dive: Troubleshooting



phase 1: Clium 으로 모든 걸 하려다 실패

- **이슈**: Cilium NetworkPolicy로 L7(HTTP Path) 까지 제어하려다가 **/auth** 경로 누락으로 **403 Forbidden** 발생
- **과정**: **이슈 : API 추가 할때 마다 Cilium + Istio 둘다 수정**
- **문제:** API가 늘어날때 마다 CIlium과 Istio 정책을 이중으로 수정해야하는 **유지보수 지옥을 경험**
- **Insight:** “Cilium은 패킷(IP/Port), Istio는 헤더(Host/Path) 역할 ”
- **결정:** 각 도구의 장점에 집중 하기로 결정 → **계층 별 역할 분담**
  - **Cilium : web → Was:8080 (L3/L4만)**
  - **Istio : /api, /auth (L7만)**

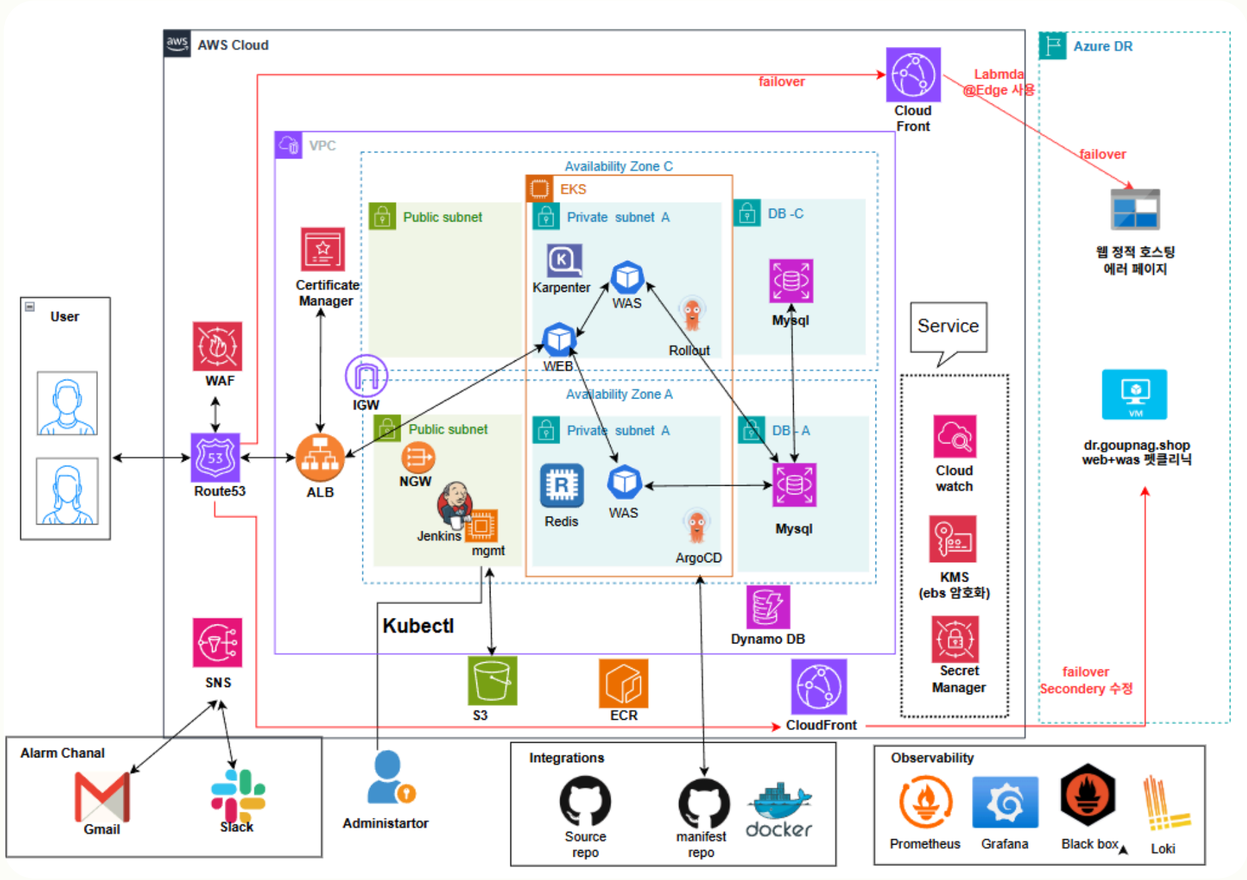
phase 2: Gateway 이슈 해결

- **문제:** L3/L4 **Host Header 구분이 불가능**해, 외부 사용자(Blog)와 내부 관리자 (Monitoring,argocd) 분리가 안됨
- **해결:**
  - ingress Gateway의 L3 제어 (Cilium)를 제거하고 **Istio AuthorizationPolicy**로 이관
  - **externalTrafficPolicy:** Local 설정으로 원본 IP 보존
  - **Blog**: Allow All/ **Monitoring:** Allow Internal IP Only 정책 적용

phase 3: 최종 결론

- **L3/L4 Security :** Cilium (Kernel Level Blocking)
- **L7 Security:** Istio (Application Level Control)
- **효과:**
  - **보안 정책 관리 효율 화 및 Host 기반 정교한 접근제어 제공**

Project: Multi-Cloud DR Architecture



Primary Site : 고가용성 AWS 아키텍처

- **AZ 장애 대응** : EKS와 RDS를 다중 가용영역에 분산 배치하여 **단일 존 장애 즉시 복구**
- **Node 장애 대응: Toplogy Spread Constaints** 설정으로 Node 장애시 Pod 균일 분산
- **Auto Scaling**: Karpenter & HPA 도입으로 트래픽 급증시 리소스 자동 스케일링
- **Security**: WAF & Private Subnet 그리고 ECR 취약점 검색 등 **외부 공격 차단 및 내부 리소스 보호**

DR Site: 비용 효율화 Azure

- **Cold Standby** : 평상 시에는 Azure 리소스를 생성하지 않고 Terraform 코드화
- **장애 발생 : Route53 Failover** 를 통해 **Blob 장애 대응 페이지** 추후에 Terraform 코드로 Azure VM 인프라 생성 **(RTO 30 분)**
- **효과:** 대기 비용을 10$ 미만 가격만을 보는 게 아닌 RTO 30분 달성

GitOps 완전 도입

- **Jenkins & ArgoCD : SSOT (Single Source Of Truh)** 달성 코드만이 진실
- **무중단 배포** : Argo Rollout Canary 배포 사용 → 무중단 배포
- **정확한 트래픽 조절 : ALB Traffic Routing** 으로 정확한 가중치 조절 가능
- **효과: 인프라 변경 이력 100% Git 추적 , 재현 가능한 인프라 , 무중단 배포**

모니터링

- **Cloud Watch** : Cloud Watch AWS 리소스 메트릭 모니터링
- **PLG stack** : 상세 메트릭 까지 모니터링 (Redis, Backup, API Health check)
- **OpenCost** : 실시간 Kubernetes 리소스 비용 추적

DR 아키텍처 트러블 슈팅 및 성과 보고서

이슈 정리

- **사건: Cloud Front Orgin Group**을 활용한 자동 Failover 구현 중, **Post/Put/Delete 요청이** 2차 오리진으로 넘어가지 않는 현상 발견
- **증거:** AWS 공식 문서 확인 결과 Origin Group Failover는 **읽기 전용 (GET/HEAD/OPTIONS)** 메서드만 지원함 (WAS 핵심 기능 불가)

조치 및 해결

[정상 상태]

www.goupang.shop → CloudFront Distribution

└── Origin: AWS ALB (EKS)

[AWS 장애 시 - Manual Failover]

1. CloudFront Distribution 설정 변경:

Origin: AWS ALB → Azure VM Public IP

2. 사용자 접속:

www.goupang.shop (동일 도메인 유지!)

→ CloudFront Edge → Azure VM

RTO

- **목표 : 30분 이내**
- **결과 : 28분 달성**
  - Terraform 인프라 활성화 (20분) + CloudFront 전파(5분) + 여유 분
- **확인:** curl -X POST 테스트를 통해 쓰기 작업 정상 동작 확인(HTTP 200 OK)

비용 절감

- **결과: 98% 절감 ($**500 → $10/월)
- **결과 :** Warm standby 대신 **Cold Standby(backup & Restore) + CloudFront 수동 변경**으로 평상시 비용을 '0'에 수렴하게 설계

교훈

- **클라우드 서비스 제약 사항은 PoC 단계에서 반드시 검증해야한다.**
  - **CloudFront Origin Group**의 한계
  - 아키텍처 설계 원칙
  - 자동 과 수동 Failover의 차이